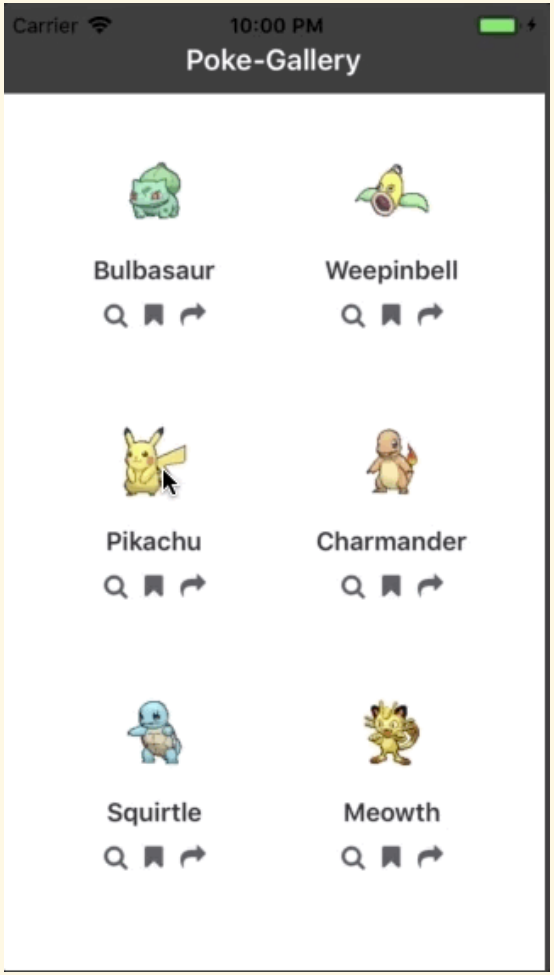
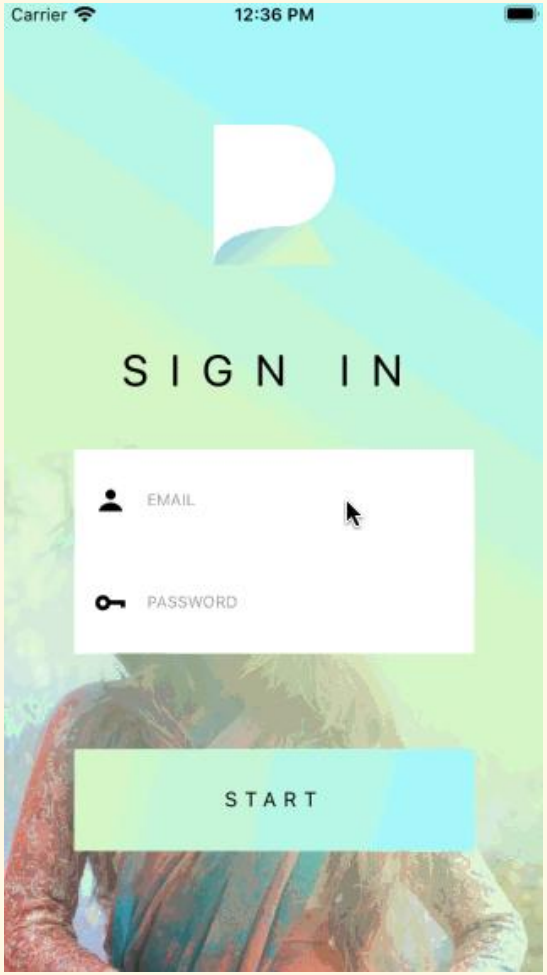
Animations

Interaction

- Animations are important for good UX





Simple Self Implementation

```
function FadeInView({children}) {
  const [opacity, setOpacity] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setOpacity(o => {
        if ((o += 0.01) > 1) {
          o = 1;
          clearInterval(id);
        }
        return o;
      });
    }, 50)
  }, []);

  return <View style={{backgroundColor: "#77f",
            opacity: opacity}}>{children}</View>
}
```

Problems:

- Not optimized (component re-rendering)
- Transition control is hard

`Animated` API

- Animatable version of common components (`View`, `Text`, `Image`, `ScrollView`, `FlatList`, `SectionList`)
  - For example: use `Animated.View` in place of `View`
- User-defined animatable components using `Animated.createAnimatedComponent()`

- Animation types:
  - Basic types: `decay`, `timing`, `spring`
  - Their compositions

Example

```
function FadeInView({children}) {
  const opacity = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(opacity,
      { toValue: 1, duration: 5000 }).start();
  }, []);

  return <Animated.View style={{ backgroundColor: "#77f",
        opacity: opacity }}>{children}</Animated.View>
}
```

- The component is not re-rendered, but only its opacity changes
- Add logs to previous examples to make sure the rendering is optimized

`Animated.timing(value, config)`

- Animates a value along a timed easing curve

- Configuration options:
  - `toValue`: Target value
  - `duration`: Length of animation in ms, default = 500
  - `delay`: Delay of start in ms, default = 500
  - `easing`: Curve easing function, default = `Easing.inOut(Easing.ease)`

Easing Functions

- Basic functions
  - `Easing.ease`
  - `Easing.linear`
  - `Easing.quad`
  - `Easing.cubic`
  - `Easing.bezier(x1, y1, x2, y2)`
  - `Easing.elastic(bounciness)`
  - `Easing.bounce`
  - …
- Composing functions
  - `Easing.in(fn)`
  - `Easing.out(fn)`
  - `Easing.inOut(fn)`

Example

```
function FadeInView({children}) {
  const opac = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(opac,
      { toValue: 1, duration: 5000,
      easing: Easing.inOut(Easing.elastic(2)) }).start();
  }, [])

  return <Animated.View style={{ backgroundColor: "#77f",
        opacity: opac }}>{children}</Animated.View>
}
```

Exercises

1. Create two animated components with different easing functions to see their different transitions.
2. Modify the components to make the animations start when a button is pressed.

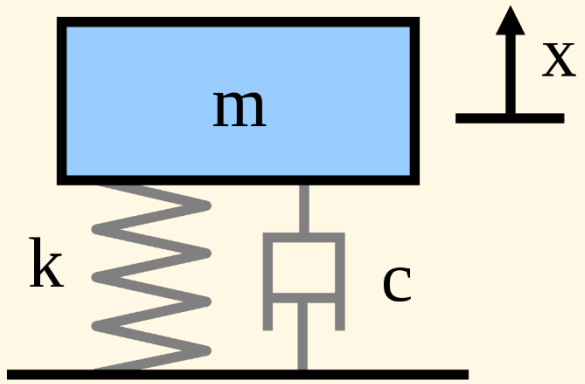
`Animated.decay(value, config)`

- Animates a value from an initial velocity to zero based on a decay coefficient

- Configuration options:
  - `velocity`: Initial velocity, required, default = 500
  - `deceleration`: Rate of decay, default = 0.997

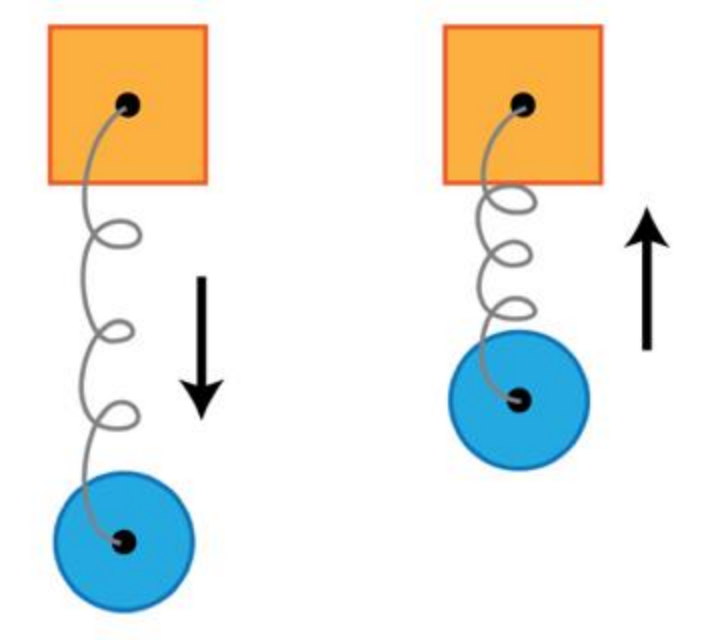
Animated.spring(value, config)

- Animates a value according to an analytical spring model based on damped harmonic oscillation
- Configuration options: use one of 3 models
  - Spring model with friction & tension:
    - `friction`: default = 7
    - `tension`: default = 40
  - Spring model with bounciness & speed:
    - `speed`: default = 12
    - `bounciness`: default = 8
  - Analytical spring model:
    - `stiffness`: default = 100
    - `damping`: default = 10
    - `mass`: default = 1
  - Common options:
    - `toValue`: Target value
    - `velocity`: Initial velocity, default = 0
    - `restSpeedThreshold`: Speed the spring is considered at rest in pixels/s, default = 0.001



Exercise

- Create a component with animated y-position on appearance using spring type



Animations Composing

- `Animated.delay(time)`
- `Animated.sequence(animations)`
- `Animated.parallel(animations, config?)`
  - `config.stopTogether`: Whether one stopped animation makes all be stopped, default = true
- `Animated.loop(animations, config?)`
  - `config.iterations`: Number of looping times, default = –1 (infinite)

Example: Delayed Animation

```
Animated.timing(opacity, {
   toValue: 1,
   duration: 5000,
   delay: 500
});
```

```
Animated.sequence([
   Animated.delay(500),
   Animated.timing(opacity, {
      toValue: 1,
      duration: 5000
   })
]);
```

Value Composing

- `Animated.add(a, b)`
- `Animated.subtract(a, b)`
- `Animated.divide(a, b)`
- `Animated.multiply(a, b)`

- `Value.interpolate({ inputRange, outputRange })`
  - Values accept numbers, angles, colors
  - Extrapolation possible with optional settings: `extrapolate`, `extrapolateLeft`, `extrapolateRight`

Example

```
<Text style={{
  opacity: value,
  top: Animated.multiply(200, value),
          // 200*value does not work
  color: value.interpolate({
    inputRange: [0, 0.2, 0.8, 1],
    outputRange: ['red','#7ff','#000','blue']
  })
}}>…</Text>
```

Exercises

1. Make the spring animation in the previous exercise looping forever
2. Create a component with two animated properties in parallel starting on its appearance:
   - y-position using spring type
   - Opacity using timing type

Animation Control

- Control functions:
  - `start(completionCallback?)`
    - `completionCallback({finished})`: callback function called after the animation finished or was stopped
  - `stop()`: Stops started animations but keep their states
  - `reset()`: Stops started animations and reset their states

Example: Pause/Resume an Animation

```
function App() {
  const top = useRef(new Animated.Value(0)).current;
  const anim = useRef(Animated.timing(top,
                 { toValue: 200, duration: 20000 })).current;
  const paused = useRef(false);

  useEffect(() => anim.start(), []);

  return <>
    <Animated.View style={ {position: 'relative', top: top} }>
      <Text>Hello</Text>
    </Animated.View>

    <Button title="Start/Stop" onPress={() => {
      if (paused.current) anim.start();
      else anim.stop();
      paused.current = ! paused.current;
    }} />
  </>
}
```

Listening to Current Animation Value

- Animation value may only be known in the native runtime due to optimizations
- Subscribing to value changes (asynchronously):
  - `Value.addListener(callback)`

- Example:
  - ```
    const value = useRef(
      new Animated.Value(0)).current;

    useEffect(() => {
      Animated.timing(value, {…}).start();
      value.addListener(v => console.log(v));
    }, []);
    ```
    ```

Exercise

- Show the current opacity of the fading component in another text component.

Animated.ValueXY

- 2D Value for driving 2D animations, mostly identical API to `Animated.Value` but multiplexed
- `ValueXY.getLayout()`: maps `{x, y}` to `{left, top}`
- Example:

```
function SpringComp() {
  const pos = useRef(
          new Animated.ValueXY({x:0, y:0})).current;

  useEffect(() => {
    Animated.spring(pos, {
      toValue: {x: 50, y: 200},
      friction: 0.1,
      tension: 0.1
    }).start();
  }, []);

  return <Animated.View style=
            {[styles.text, pos.getLayout()]}>
    <Text>Hello</Text>
  </Animated.View>
}
```